

Schedule (Phases)

Split I: Backend Development

Phase 1: Authentication Foundation — COMPLETED

Status: Completed

Goal:

Build the first backend foundation for user authentication and persistent account data.

Completed/Included:

- UserAccount model
- AuthToken model
- Login DTOs / request-response objects
- AuthController
- AuthService
- Repository pattern setup
- EF Core DbContext setup
- PostgreSQL connection
- Initial migration
- Update database
- Basic password hashing
- Token generation
- Token validation
- Login endpoint tested through Postman

Phase 2: Relationship/Pond Foundation

Status: Not started

Goal:

Create the private relationship space that all PondLite data belongs to.

Planned:

- Relationship model
- RelationshipMember model
- Relationship creation logic
- Relationship membership lookup
- Connect user accounts to a Relationship
- Support one active Pond/Relationship for PondLite v1
- Add authorization helper logic to confirm a user belongs to a Relationship
- Add basic Create/Join Pond endpoint or temporary manual/seeder setup
- Update database with new migration

Important Design Notes:

- Relationship is the backend/domain term.

- Pond is the user-facing term.
- A Relationship should normally have exactly two RelationshipMembers.
- A user should only access data belonging to a Relationship they are a member of.

Phase 3: Frog Foundation

Status: Not started

Goal:

Create the frog identity and state system for each RelationshipMember.

Planned:

- Frog model
- One Frog per RelationshipMember
- Frog name
- CurrentMood
- ActivityState
- LastUpdatedAt
- Frog creation when a RelationshipMember is created
- Basic frog retrieval endpoint
- Basic frog update logic
- Update database with new migration

Important Design Notes:

- Frogs act as emotional avatars for PondMates.
- Frogs should appear asleep/inactive until the user completes the daily check-in.
- Frog state should be private to the Relationship/Pond.

Phase 4: Daily Check-In

Status: Not started

Goal:

Allow users to complete one emotional check-in per day inside their Pond.

Planned:

- DailyCheckIn model
- Emotion enum
- PrimaryEmotion, SecondaryEmotion, TertiaryEmotion
- OptionalMessage
- CheckInDate
- Create daily check-in endpoint
- Retrieve today's check-in status
- Retrieve both PondMates' current check-in states
- Update frog mood/activity state after check-in
- Prevent duplicate check-ins for the same user/date/relationship, or allow controlled editing
- Add unique constraint for RelationshipId + UserAccountId + CheckInDate
- Update database with new migration

Important Design Notes:

- A user can have many DailyCheckIns over time.
- A user should only have one DailyCheckIn per day per Relationship.
- Optional messages should only be visible to the PondMate in the same Relationship.

Phase 5: Ribbits / Gentle Reminders

Status: Not started

Goal:

Create a lightweight way for PondMates to send thoughts, care offers, gentle requests, or little nudges.

Planned:

- Ribbit model
- RibbitType enum
 - Thought
 - CareOffer
 - GentleRequest
 - LittleNudge
- RibbitStatus enum
 - Sent
 - Acknowledged
 - Completed
 - Declined
 - Cancelled
- SenderUserId
- ReceiverUserId
- RelationshipId
- Message
- AcknowledgementEmoji
- DeliveryTime
- CreatedAt
- AcknowledgedAt
- CompletedAt
- DeclinedAt
- Create/send Ribbit endpoint
- Retrieve active Ribbits for user
- Acknowledge Ribbit endpoint
- Complete/decline Ribbit endpoint where applicable
- Cancel Ribbit endpoint for sender
- Update frog notification indicator when unread/unacknowledged Ribbits exist
- Update database with new migration

Important Design Notes:

- Ribbits should feel softer than tasks.
- Sender and receiver must both belong to the same Relationship.
- Ribbits should never be visible outside the shared Pond.

Phase 6: Discussion Prompts

Status: Not started

Goal:

Add optional daily relationship-oriented discussion prompts.

Planned:

- DiscussionPrompt model
- DiscussionResponse model
- PromptDate
- PromptText
- ResponseText
- Create/retrieve daily prompt endpoint
- Submit discussion response endpoint
- Retrieve responses for the current prompt
- Track completion status, such as 0/2, 1/2, or 2/2 responses
- Add unique constraint for DiscussionPromptId + UserAccountId
- Update database with new migration

Important Design Notes:

- Discussion prompts should be optional.
- No punishment should occur for skipping prompts.
- Responses should only be visible to PondMates in the same Relationship.
- Prompts may initially come from a stored/static list before AI generation is added.

Phase 7: Bug-Themed Gift System

Status: Not started

Goal:

Create a playful lightweight reward/gift system that supports connection without pressure.

Planned:

- Bug stash/balance tracking
- Bug reward when both PondMates complete their daily check-in
- Feed/gift bug endpoint
- Track when a frog was fed by PondMate
- Display bug count
- Display frog reaction when fed
- Optional bug transaction/history model if needed

Possible Model:

- BugBalance

- Id
- RelationshipMemberId
- CurrentBugs
- UpdatedAt

Optional Model:

- BugTransaction
 - Id
 - RelationshipId
 - SenderUserId
 - ReceiverUserId
 - Amount
 - Reason
 - CreatedAt

Important Design Notes:

- Bugs should reward shared engagement without creating guilt.
- Bugs should support the feeling that both users are caring for the Pond together.
- This should not become a pressure/streak system.

Phase 8: Smart Connection Assistant

Status: Not started

Goal:

Add optional AI-generated suggestions for care actions, message ideas, and discussion support.

Planned:

- SmartConnectionSuggestion model
- AI request DTO
- AI response DTO
- Request suggestion endpoint
- Gather minimum required Relationship/Pond context
- Include current mood/check-in context when available
- Include recent Ribbit/discussion context when useful
- Send prepared context to AI service
- Receive and store suggestion
- Allow user to accept, ignore, or regenerate suggestion
- Track Accepted status
- Add optional references to DailyCheckIn, Ribbit, and DiscussionPrompt where useful
- Update database with new migration

Important Design Notes:

- AI suggestions should be optional and user-controlled.
- AI should not automatically send messages, complete Ribbits, or make decisions.

- The system should only send the minimum necessary context to the AI service.
- Suggestions should remain private to the requesting user unless the user chooses to act on them.
- AI should not be framed as therapy or counseling.

Phase 9: Backend Privacy and Authorization Pass

Status: Not started

Goal:

Review the backend to make sure every protected feature enforces Relationship/Pond access rules.

Planned:

- Review all protected endpoints
- Confirm user identity from token/session
- Confirm RelationshipMember access before returning private data
- Deny access by default if membership cannot be verified
- Ensure all Relationship-owned data uses RelationshipId
- Ensure users cannot access another Relationship's check-ins, Ribbits, prompts, responses, frogs, or AI suggestions
- Add consistent error responses for unauthorized access
- Add backend validation for sender/receiver membership
- Add tests in Postman for unauthorized access attempts

Important Design Notes:

- Authentication answers "Who is this user?"
- RelationshipMember authorization answers "What Pond can this user access?"
- Privacy should be treated as a core backend rule, not a frontend-only concern.

Split II: Frontend Development

Phase 1: Frontend Project Setup

Status: Not started

Goal:

Create the initial frontend structure for PondLite.

Planned:

- Choose frontend approach
 - React/PWA recommended for early version
 - React Native, Expo, .NET MAUI, or Blazor can be considered later
- Create frontend project
- Set up routing
- Set up shared style/theme files
- Set up API service layer

- Set up session/localStorage handling
- Create basic page layout
- Create basic navigation

Important Design Notes:

- Frontend should stay mobile-friendly from the beginning.
- Even if it starts as a web app, it should be designed with phone usage in mind.

Phase 2: Login, Logout, and Account Pages

Status: Not started

Goal:

Connect the frontend to the authentication backend.

Planned:

- Login page
- Account creation page
- Logout button
- Store token/session information
- Redirect logged-in users to Pond/Home
- Redirect logged-out users to Login
- Show login/session errors
- Clear session on logout
- Handle expired token/session

Phase 3: Pond/Home Page

Status: Not started

Goal:

Create the main Pond screen where users see the shared relationship space.

Planned:

- Pond page layout
- Display both frogs
- Display current frog state
- Display check-in status
- Display bug count or simple reward indicator
- Display Ribbit notification indicator
- Display daily discussion prompt area
- Display Smart Connection Assistant entry point
- Handle no PondMate/no Relationship state

Important Design Notes:

- This page should feel soft, safe, and playful.
- The Pond should not feel like a dashboard full of chores.

Phase 4: Daily Check-In UI

Status: Not started

Goal:

Allow the user to complete their daily emotional check-in.

Planned:

- Check-in prompt/modal
- Emotion wheel or emotion selection UI
- Primary emotion required
- Secondary and tertiary emotions optional
- Optional message to PondMate
- Submit check-in
- Show check-in completion state
- Update frog display after check-in

Phase 5: Ribbit UI

Status: Not started

Goal:

Allow PondMates to send and respond to Ribbits.

Planned:

- Create Ribbit form
- Select Ribbit type
- Enter message
- Optional delivery time
- View active Ribbits
- Acknowledge with emoji
- Complete or decline applicable Ribbits
- Cancel sent Ribbits
- Show soft notifications on frog/Pond screen

Phase 6: Discussion Prompt UI

Status: Not started

Goal:

Allow users to view and answer optional daily prompts.

Planned:

- Show daily discussion prompt
- Response input
- Submit response
- Edit response if allowed
- Show 0/2, 1/2, or 2/2 completion status
- Display PondMate response when appropriate
- Allow skipping without negative messaging

Phase 7: Bug Gift/Frog Reaction UI

Status: Not started

Goal:

Create a simple playful gift interaction.

Planned:

- Display bug stash
- Feed/gift bug button
- Show frog reaction when fed
- Show notification that PondMate fed the frog
- Keep interaction lightweight and fun

Phase 8: Smart Connection Assistant UI

Status: Not started

Goal:

Allow users to request optional AI support.

Planned:

- Smart Connection Assistant button/panel
- Display generated care action
- Display generated message suggestion
- Display generated discussion prompt suggestion
- Accept suggestion
- Ignore suggestion
- Regenerate suggestion
- Explain that suggestions are optional

Phase 9: Frontend Polish and Mobile Responsiveness

Status: Not started

Goal:

Make PondLite feel pleasant and usable on phone-sized screens.

Planned:

- Responsive layout
- Soft visual style
- Consistent buttons/cards
- Frog/pond visual polish
- Friendly empty states
- Loading states
- Error states
- Accessibility basics
- Final pass for low-pressure wording

Split III: Private Deployment to Raspberry Pi

Phase 1: Raspberry Pi / Umbrel Planning

Status: Not started

Goal:

Plan the private deployment environment before moving the app off the dev machine.

Planned:

- Decide whether to use Umbrel directly or a normal Raspberry Pi OS/server setup
- Confirm Raspberry Pi hardware
- Confirm storage approach
- Confirm network setup
- Confirm whether PostgreSQL, API, or both will run on the Pi
- Decide backup approach
- Document private deployment assumptions

Important Design Notes:

- This deployment is only for Brandon and Michelle/private trusted testing.
- This is not the public production architecture.

Phase 2: Tailscale Private Network Setup

Status: Not started

Goal:

Allow secure remote access without exposing the app/database directly to the public internet.

Planned:

- Install Tailscale on Raspberry Pi
- Install Tailscale on Brandon's devices
- Install Tailscale on Michelle's device
- Confirm devices can reach each other through Tailscale
- Record Raspberry Pi Tailscale address
- Avoid public port forwarding
- Avoid exposing PostgreSQL directly to the public internet

Phase 3: PostgreSQL Deployment on Raspberry Pi

Status: Not started

Goal:

Move database hosting from local dev machine to Raspberry Pi.

Planned:

- Install/configure PostgreSQL on Raspberry Pi
- Create PondLite database
- Create database user

- Configure database permissions
- Update connection string for private environment
- Run EF Core migrations against Pi database
- Test login/account persistence with Pi database
- Document database backup process

Phase 4: ASP.NET Core API Deployment on Raspberry Pi

Status: Not started

Goal:

Host the PondLite backend API on the Raspberry Pi.

Planned:

- Publish ASP.NET Core API
- Deploy API to Raspberry Pi
- Run API as a service or container
- Configure appsettings/environment variables
- Connect API to PostgreSQL
- Test API through Tailscale address
- Confirm authentication works remotely
- Confirm protected endpoints work remotely

Phase 5: Private End-to-End Testing

Status: Not started

Goal:

Let Brandon and Michelle test the private PondLite version away from the local dev environment.

Planned:

- Connect frontend to Raspberry Pi API through Tailscale
- Test login/logout
- Test Pond access
- Test daily check-in
- Test Ribbits
- Test discussion prompt
- Test bug/frog interactions if implemented
- Test from phone/laptop
- Confirm no public internet exposure
- Document known issues

Phase 6: Private Deployment Maintenance

Status: Not started

Goal:

Make the private deployment stable enough for early real use.

Planned:

- Basic database backups
- Simple restore process
- Log review
- Update process for backend changes
- Update process for database migrations
- Tailscale device management
- Document how to restart services
- Document what to do if home internet/power goes down

Split IV: Public Deployment to Azure

Phase 1: Public Deployment Planning

Status: Future

Goal:

Plan the production architecture before exposing PondLite to real public users.

Planned:

- Decide final frontend/mobile approach
- Decide Azure hosting approach
- Decide managed database provider
- Decide storage needs
- Decide domain/API URL structure
- Review authentication strategy
- Review privacy and data deletion requirements
- Estimate monthly cost
- Decide what must be completed before App Store/Play Store release

Phase 2: Azure Backend Hosting

Status: Future

Goal:

Deploy the ASP.NET Core API to a public cloud environment.

Planned:

- Deploy API to Azure App Service or container hosting
- Configure production environment variables
- Configure HTTPS
- Configure logging
- Configure app settings/secrets
- Test public API access
- Confirm protected endpoints require authentication

Phase 3: Managed PostgreSQL Database

Status: Future

Goal:

Move production data to managed cloud database hosting.

Planned:

- Create managed PostgreSQL database
- Configure secure connection
- Run EF Core migrations
- Configure backup/restore
- Configure access restrictions
- Test production database persistence
- Plan migration path from private Pi database if needed

Phase 4: Production Authentication and Security Improvements

Status: Future

Goal:

Improve security before public users are allowed.

Planned:

- Review manual auth implementation
- Consider ASP.NET Core Identity or another production-ready auth approach
- Add password reset
- Add email verification
- Improve token/session management
- Add rate limiting
- Add account lockout or brute-force protection if appropriate
- Ensure secure password hashing
- Ensure HTTPS-only production access
- Review CORS policy
- Review secrets management

Phase 5: Production Privacy Requirements

Status: Future

Goal:

Prepare PondLite for public privacy expectations.

Planned:

- Privacy policy
- Terms of service
- Account deletion
- Data deletion
- Export/download user data if appropriate
- Limit AI data sharing
- Clearly disclose AI usage

- Review logging to avoid storing sensitive relationship content unnecessarily
- Add admin/support boundaries if support tools are ever added

Phase 6: Monitoring, Backups, and Reliability

Status: Future

Goal:

Make the production system maintainable.

Planned:

- Application monitoring
- Error logging
- Database backups
- Restore testing
- Uptime checks
- Deployment rollback plan
- Basic cost monitoring
- Performance review
- Scaling plan

Phase 7: App Store / Play Store Readiness

Status: Future

Goal:

Prepare PondLite for mobile/public release.

Planned:

- Final mobile app or PWA decision
- App Store / Play Store developer accounts
- App privacy disclosures
- Screenshots and listing copy
- Support/contact information
- Production API URL
- Production database
- Final auth/privacy review
- Beta testing
- Release checklist

Later / Post-v1 Feature Ideas

These features should remain outside the core PondLite v1 scope unless the app becomes stable enough to expand.

Possible Later Features:

- Pond decorations

- Frog interactions
- Wandering frogs
- Sound
- Child frogs
- Marry feature
- Pond Rhythm Rewards
- Bonus bugs or small visual flair for consistent shared use
- More advanced AI personalization
- Memory/archive features
- Photo memories
- Retired/archived frogs
- Seasonal Pond reflections
- Expanded prompt packs
- More frog customization

Important Design Note:

Later reward features should remain non-punitive. PondLite should not remove progress, shame users, punish missed days, or create guilt-based pressure. Any consistency rewards should be framed as gentle Pond activity or shared rhythm rather than streaks that reset to zero.